

ECE 60146 HW 05

Andrea Goh

February 2025

1 Task: Deeper Networks

In this assignment, the goal is to compare the performance of three different networks. The first two networks are taken directly from **DLStudio.ExperimentsWithCIFAR**, the first having two convolutional networks and the second having three. The third network used is a deeper network with eight convolutional layers. A subclass called **CustomExperimentsWithCIFAR** is created, which inherits from **DLStudio**. Within this subclass, the third network called **Net3** is defined as a subclass. The architecture of **Net3** has eight convolutional layers only two pooling layers. Performance deprecited when more than two pooling layers are used.

```
1 class CustomExperimentsWithCIFAR(DLStudio):
2     def __init__(self):
3         super().__init__(
4             dataroot = "./data/CIFAR-10/",
5             image_size = [32,32],
6             path_saved_model = "./saved_model",
7             momentum = 0.9,
8             learning_rate = 1e-3,
9             epochs = 10,
10            batch_size = 4,
11            classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', '
12                    ship', 'truck'),
13            use_gpu = True,
14        )
15
16 class Net3(nn.Module):
17     def __init__(self):
18         super(CustomExperimentsWithCIFAR.Net3, self).__init__()
19         self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
20         self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
21         self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
22         self.conv4 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
23         self.conv5 = nn.Conv2d(256, 512, kernel_size=3, padding=1)
24         self.conv6 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
25         self.conv7 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
26         self.conv8 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
27
28         self.fc1 = nn.Linear(512 * 8 * 8, 256)
29         self.fc2 = nn.Linear(256, 10)
30         self.pool = nn.MaxPool2d(2,2)
31         self.dropout = nn.Dropout(0.5)
32
33     def forward(self, x):
34         x = F.relu(self.conv1(x))
35         x = F.relu(self.conv2(x))
36         x = self.pool(F.relu(self.conv3(x)))
37         x = F.relu(self.conv4(x))
38         x = F.relu(self.conv5(x))
39         x = self.pool(F.relu(self.conv6(x)))
40         x = F.relu(self.conv7(x))
41         x = F.relu(self.conv8(x))
42         x = x.view(x.size(0), -1)
43         x = F.relu(self.fc1(x))
44         x = self.dropout(x)
45         x = self.fc2(x)
46         return x
```

2 Results

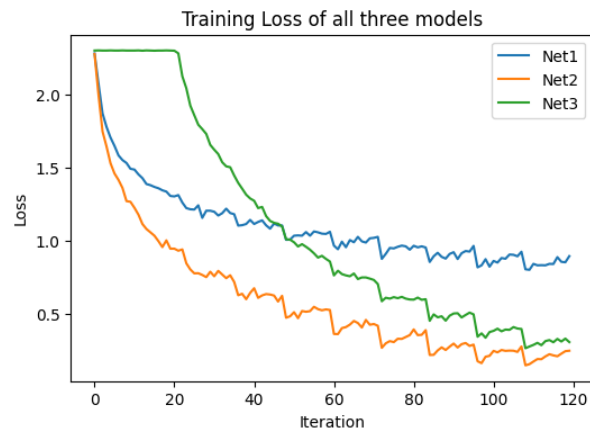
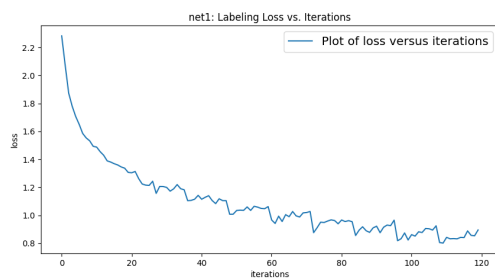
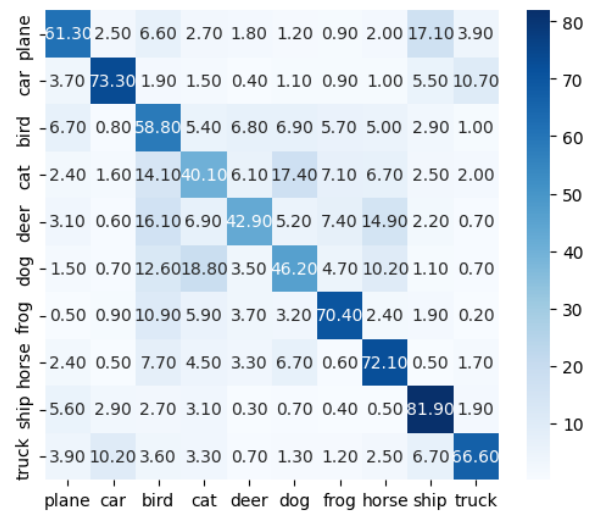


Figure 1: Training Loss for all Networks

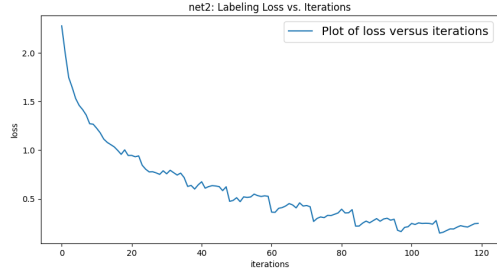


(a) Net1: Loss vs. Iterations

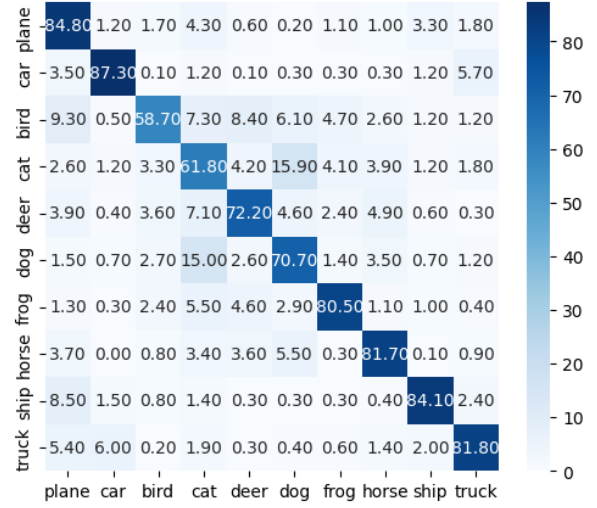


(b) Net1: Confusion Matrix

Figure 2: fig: Results from Net1

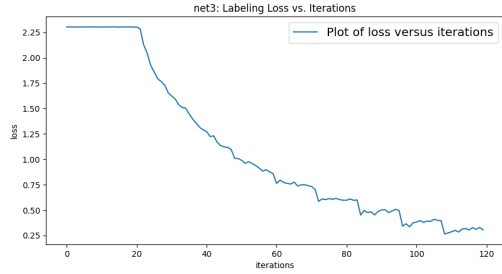


(a) Net2: Loss vs. Iterations

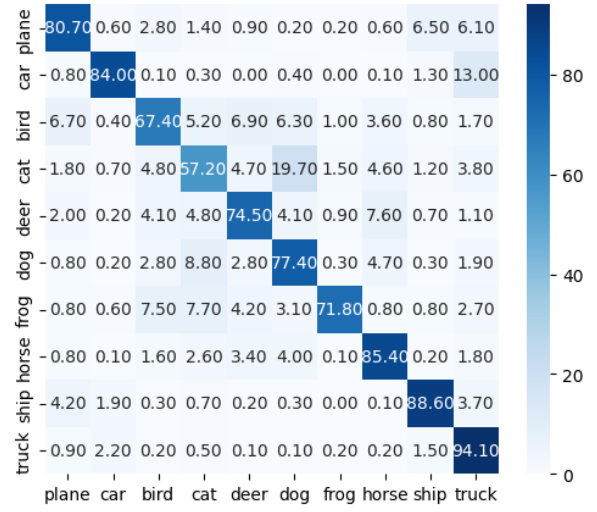


(b) Net2: Confusion Matrix

Figure 3: fig: Results from Net2



(a) Net3: Loss vs. Iterations



(b) Net3: Confusion Matrix

Figure 4: fig: Results from Net3

Network	Net1	Net2	Net3
Accuracy	61%	76%	78%

Table 1: Overall Accuracy of 3 Models

	Net1	Net2	Net3
plane	61%	84%	80%
car	73%	87%	84%
bird	58%	58%	67%
cat	40%	61%	57%
deer	42%	72%	74%
dog	46%	70%	77%
frog	70%	80%	71%
horse	72%	81%	85%
ship	81%	84%	88%
truck	66%	81%	94%

Table 2: Class Accuracies

2.1 Observations: Training Loss

From figure 1, we can see that the final loss value is the highest for Net1, followed by Net3, and followed by Net2. The losses for Net1 and Net2 immediately decreases, with Net2 decreasing at a faster rate. On the other hand, Net3 only starts decreasing around iteration 25.

2.2 Observations: Testing Performance

From table 1, we can see that Net1 has the worst overall performance. From figure 4, we can see that the classification of the cat class has the lowest accuracy, with 18.8% of the cat images being classified as dog. These results make sense because both dogs and cats are four legged animals, with relatively short necks, compared to horses and deer. Thus, it would be difficult to differentiate between a dog and cat, especially when the images used are of such low resolution (32×32). On the other hand, the ship class was the most accurately classified.

Net2 exhibits significant improvements from Net1, with a 15% higher accuracy. Previously in Net1, the cat class was the most poorly classified. Net2 displayed a significant improvement as the classification accuracy for cat increased by 21%. This class was still most often misclassified as dog due to the similarities of both animals.

Net3 only shows a slight improvement from Net2, with a 2% increase in accuracy. The improved performance in Net3 is only observed at a larger number of epochs. In this assignment, 10 epochs were used. Performance of Net2 is better when less than 10 epochs are used. When less than 3 epochs are used, Net3 completely fails in classification.

While Net3 had the highest accuracy, it was the slowest to train, taking 990 seconds, which is 1.6 times slower than training Net2, which took 614 seconds. The time taken to train Net2 was very similar to the time taken to train Net1, which was 570 seconds. Despite the similarities in training times, Net2 exhibited significantly higher testing accuracies. Therefore, we can justify using Net2 for this classification task.

3 Source Code

```

1 import torch
2 import seaborn as sns
3 from DLStudio import *
4
5 dls = DLStudio(
6     #
7     dataroot = "/home/kak/ImageDatasets/CIFAR-10/",
8     dataroot = "./data/CIFAR-10/",
9     image_size = [32,32],
10    path_saved_model = "./saved_model",
11    momentum = 0.9,
12    learning_rate = 1e-3,
13    epochs = 10,
14    batch_size = 4,
15    classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', '
16              ship', 'truck'),
17    use_gpu = True,
18 )

```

```

17
18 class CustomExperimentsWithCIFAR(DLStudio):
19     def __init__(self):
20         super().__init__(
21             #
22                 dataroot = "/home/kak/ImageDatasets/CIFAR-10/",
23                 dataroot = "./data/CIFAR-10/",
24                 image_size = [32,32],
25                 path_saved_model = "./saved_model",
26                 momentum = 0.9,
27                 learning_rate = 1e-3,
28                 epochs = 10,
29                 batch_size = 4,
30                 classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', '
31                             ship', 'truck'),
32                 use_gpu = True,
33         )
34
35     def run_code_for_training(self, net, net_name, display_images=False):
36         filename_for_out = "performance_numbers_" + net_name + str(self.epochs) + ".txt"
37         FILE = open(filename_for_out, 'w')
38         net = copy.deepcopy(net)
39         net = net.to(self.device)
40         criterion = nn.CrossEntropyLoss()
41         optimizer = optim.SGD(net.parameters(), lr=self.learning_rate, momentum=self.
42                                 momentum)
43         print("\n\nStarting training loop...")
44         start_time = time.perf_counter()
45         loss_tally = []
46         elapsed_time = 0.0
47         for epoch in range(self.epochs):
48             print("")
49             running_loss = 0.0
50             for i, data in enumerate(self.train_data_loader):
51                 inputs, labels = data
52                 if i % 1000 == 999:
53                     current_time = time.perf_counter()
54                     elapsed_time = current_time - start_time
55                     print("\n\nEpoch:%d/%d\niter=%4d\nelapsed_time=%5d\nsecs]\n\nGround
56                             Truth:UUUUU" %
57                             (epoch+1, self.epochs, i+1, elapsed_time) +
58                             '\n'.join('%10s' % self.class_labels[labels[j]] for j in range(
59                                 self.batch_size)))
60                     inputs = inputs.to(self.device)
61                     labels = labels.to(self.device)
62                     ## Since PyTorch likes to construct dynamic computational graphs, we
63                     need to
64                     ## zero out the previously calculated gradients for the learnable
65                     parameters:
66                     optimizer.zero_grad()
67                     outputs = net(inputs)
68                     loss = criterion(outputs, labels)
69                     running_loss += loss.item()
70                     if i % 1000 == 999:
71                         _, predicted = torch.max(outputs.data, 1)
72                         print("[epoch:%d/%d\niter=%4d\nelapsed_time=%5d\nsecs]\n\nPredicted
73                                 Labels:U" %
74                                 (epoch+1, self.epochs, i+1, elapsed_time) + '\n'.join('%10s' % self
75                                     .class_labels[predicted[j]] for j in range(self.batch_size)))
76                     avg_loss = running_loss / float(1000)
77                     loss_tally.append(avg_loss)
78                     print("[epoch:%d/%d\niter=%4d\nelapsed_time=%5d\nsecs]\n\nLoss:U%.3f"
79                             % (epoch+1, self.epochs, i+1, elapsed_time, avg_loss))
80                     FILE.write("%.3f\n" % avg_loss)
81                     FILE.flush()
82                     running_loss = 0.0
83                     if display_images:
84                         logger = logging.getLogger()
85                         old_level = logger.level
86                         logger.setLevel(100)
87                         plt.figure(figsize=[6,3])
88                         plt.imshow(np.transpose(torchvision.utils.make_grid(inputs,
89                                     normalize=False, padding=3, pad_value=255).cpu(), (1,2,0)))
89                         plt.savefig(f'TrainingImages_{net_name}.png')

```

```

80         logger.setLevel(old_level)
81         loss.backward()
82         optimizer.step()
83     print("\nFinished Training\n")
84     self.save_model(net)
85     plt.figure(figsize=(10,5))
86     plt.title(net_name + ": Labeling Loss vs. Iterations")
87     plt.plot(loss_tally)
88     plt.xlabel("iterations")
89     plt.ylabel("loss")
90     plt.legend(["Plot of loss versus iterations"], fontsize="x-large")
91     plt.savefig(f"loss_vs_iterations_{net_name}.png")
92
93 def run_code_for_testing(self, net, net_name, display_images=False):
94     net.load_state_dict(torch.load(self.path_saved_model))
95     net = net.eval()
96     net = net.to(self.device)
97     filename_for_results = "classification_results_" + net_name + str(self.epochs) +
98         ".txt"
99     FILE = open(filename_for_results, 'w')
100     correct = 0
101     total = 0
102     confusion_matrix = torch.zeros(len(self.class_labels), len(self.class_labels))
103     class_correct = [0] * len(self.class_labels)
104     class_total = [0] * len(self.class_labels)
105     with torch.no_grad():
106         for i, data in enumerate(self.test_data_loader):
107             ## data is set to the images and the labels for one batch at a time:
108             images, labels = data
109             images = images.to(self.device)
110             labels = labels.to(self.device)
111             if i % 1000 == 999:
112                 print("\n\n[i=%d:] Ground Truth: " % (i+1) + ', '.join('%5s' %
113                     self.class_labels[labels[j]] for j in range(self.batch_size)))
114                 outputs = net(images)
115                 _, predicted = torch.max(outputs.data, 1)
116                 if i % 1000 == 999:
117                     print("[i=%d:] Predicted Labels: " % (i+1) + ', '.join('%5s' % self.
118                         class_labels[predicted[j]] for j in range(self.batch_size)))
119                     logger = logging.getLogger()
120                     old_level = logger.level
121                     if display_images:
122                         logger.setLevel(100)
123                         plt.figure(figsize=[6,3])
124                         plt.imshow(np.transpose(torchvision.utils.make_grid(images,
125                             normalize=False, padding=3, pad_value=255).cpu(), (1,2,0)))
126                         plt.savefig(f'Testing_Images_{net_name}.png')
127                         logger.setLevel(old_level)
128                     for label, prediction in zip(labels, predicted):
129                         confusion_matrix[label][prediction] += 1
130                     total += labels.size(0)
131                     correct += (predicted == labels).sum().item()
132                     ## comp is a list of size batch_size of "True" and "False" vals
133                     comp = predicted == labels
134                     for j in range(self.batch_size):
135                         label = labels[j]
136                         class_correct[label] += comp[j].item()
137                         class_total[label] += 1
138         for j in range(len(self.class_labels)):
139             print('Prediction accuracy for %5s: %2d%%' % (self.class_labels[j], 100 *
140                 class_correct[j] / class_total[j]))
141             FILE.write('\n\nPrediction accuracy for %5s: %2d%%\n' % (self.class_labels
142                 [j], 100 * class_correct[j] / class_total[j]))
143         print("\n\nOverall accuracy of the network on the 10000 test images: %d%%" %
144             (100 * correct / float(total)))
145         FILE.write("\n\nOverall accuracy of the network on the 10000 test images: %d%%\n" %
146             (100 * correct / float(total)))
147         print("\n\nDisplaying the confusion matrix:\n")
148         FILE.write("\n\nDisplaying the confusion matrix:\n\n")
149         out_str = " "
150         for j in range(len(self.class_labels)):
151             out_str += "%7s" % self.class_labels[j]
152         print(out_str + "\n")

```

```

144     FILE.write(out_str + "\n\n")
145     for i,label in enumerate(self.class_labels):
146         out_percents = [100 * confusion_matrix[i,j] / float(class_total[i]) for j in
147                         range(len(self.class_labels))]
148         out_percents = ["%.2f" % item.item() for item in out_percents]
149         out_str = "%6s:␣" % self.class_labels[i]
150         for j in range(len(self.class_labels)): out_str += "%7s" % out_percents[j]
151         print(out_str)
152         FILE.write(out_str + "\n")
153     FILE.close()
154     self.class_correct = class_correct
155     self.class_total = class_total
156     self.correct = correct
157     self.total = total
158     self.display_conf_matrix(confusion_matrix, net_name)
159
160 def display_conf_matrix(self, confusion_matrix, net_name):
161     accuracy_matrix = []
162     for i,label in enumerate(self.class_labels):
163         out_percents = [100 * confusion_matrix[i,j] / float(self.class_total[i]) for
164                         j in range(len(self.class_labels))]
165         out_percents = [item.item() for item in out_percents]
166         accuracy_matrix.append([out_percents[j] for j in range(len(self.class_labels)
167                               )])
168     plt.figure(figsize=(6,5))
169     sns.heatmap(np.array(accuracy_matrix, dtype=float), annot=True, fmt='%.2f', cmap=
170                 'Blues', cbar=True,
171                 xticklabels=self.class_labels, yticklabels=self.class_labels)
172     plt.savefig(f'{net_name}_Confusion_matrix.png', bbox_inches='tight')
173
174 def load_cifar_10_dataset(self):
175     ## But then the call to Normalize() changes the range to -1.0-1.0 float vals.
176     transform = tvf.Compose([tvf.ToTensor(),
177                             tvf.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))]) ##
178                                     accuracy: 51%
179
180     ## Define where the training and the test datasets are located:
181     train_data_loc = torchvision.datasets.CIFAR10(root=self.dataroot, train=True,
182           download=True, transform=transform)
183     test_data_loc = torchvision.datasets.CIFAR10(root=self.dataroot, train=False,
184           download=True, transform=transform)
185
186     ## Now create the data loaders:
187     self.train_data_loader = torch.utils.data.DataLoader(train_data_loc, batch_size=
188           self.batch_size, shuffle=True, num_workers=2)
189     self.test_data_loader = torch.utils.data.DataLoader(test_data_loc, batch_size=
190           self.batch_size, shuffle=False, num_workers=2)
191
192
193 class Net3(nn.Module):
194     def __init__(self):
195         super(CustomExperimentsWithCIFAR.Net3, self).__init__()
196         self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
197         self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
198         self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
199         self.conv4 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
200         self.conv5 = nn.Conv2d(256, 512, kernel_size=3, padding=1)
201         self.conv6 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
202         self.conv7 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
203         self.conv8 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
204
205         self.fc1 = nn.Linear(512 * 8 * 8, 256)
206         self.fc2 = nn.Linear(256, 10)
207         self.pool = nn.MaxPool2d(2,2)
208         self.dropout = nn.Dropout(0.5)
209
210     def forward(self, x):
211         x = F.relu(self.conv1(x))
212         x = F.relu(self.conv2(x))
213         x = self.pool(F.relu(self.conv3(x)))
214         x = F.relu(self.conv4(x))
215         x = F.relu(self.conv5(x))
216         x = self.pool(F.relu(self.conv6(x)))
217         x = F.relu(self.conv7(x))

```

```

208         x = F.relu(self.conv8(x))
209         x = x.view(x.size(0), -1)
210         x = F.relu(self.fc1(x))
211         x = self.dropout(x)
212         x = self.fc2(x)
213         return x
214
215
216 def compare_loss():
217     with open('performance_numbers_net1.txt', 'r') as file:
218         running_loss1 = [float(line.strip()) for line in file]
219     with open('performance_numbers_net2.txt', 'r') as file:
220         running_loss2 = [float(line.strip()) for line in file]
221     with open('performance_numbers_net3.txt', 'r') as file:
222         running_loss3 = [float(line.strip()) for line in file]
223
224     plt.figure(figsize=(6,4))
225     plt.plot(running_loss1, label = 'Net1')
226     plt.plot(running_loss2, label = 'Net2')
227     plt.plot(running_loss3, label = 'Net3')
228     plt.title('Training Loss of all three models')
229     plt.xlabel('Iteration')
230     plt.ylabel('Loss')
231     plt.legend()
232     plt.savefig('Training Loss Comparison.png', bbox_inches='tight')
233     return
234
235 exp_cifar = DLStudio.ExperimentsWithCIFAR(dl_studio=dls)
236 exp_cifar_custom = CustomExperimentsWithCIFAR()
237
238 exp_cifar.load_cifar_10_dataset()
239 exp_cifar_custom.load_cifar_10_dataset()
240
241 # Instantiate models
242 model1 = exp_cifar.Net()
243 model2 = exp_cifar.Net2()
244 model3 = exp_cifar_custom.Net3()
245
246 # Display network properties
247 number_of_learnable_params = sum(p.numel() for p in model1.parameters() if p.
    requires_grad)
248 print("\n\nThe number of learnable parameters in the model: %d" %
    number_of_learnable_params)
249 number_of_learnable_params = sum(p.numel() for p in model2.parameters() if p.
    requires_grad)
250 print("\n\nThe number of learnable parameters in the model: %d" %
    number_of_learnable_params)
251 number_of_learnable_params = sum(p.numel() for p in model3.parameters() if p.
    requires_grad)
252 print("\n\nThe number of learnable parameters in the model: %d" %
    number_of_learnable_params)
253
254 # Training and Testing
255 exp_cifar_custom.run_code_for_training(model1, 'net1', display_images=False)
256 exp_cifar_custom.run_code_for_testing(model1, 'net1', display_images=False)
257
258 exp_cifar_custom.run_code_for_training(model2, 'net2', display_images=False)
259 exp_cifar_custom.run_code_for_testing(model2, 'net2', display_images=False)
260
261 exp_cifar_custom.run_code_for_training(model3, 'net3', display_images=False)
262 exp_cifar_custom.run_code_for_testing(model3, 'net3', display_images=False)
263
264 compare_loss()

```
